Read through the implementation. The details will be described in more detail below.

sql/scrub_batched_direct_updates.sql

```
CREATE OR REPLACE PROCEDURE scrub_batches()
LANGUAGE plpgsql
AS $$
DECLARE
  current_id int := (SELECT MIN(id) FROM users);
  max_id int := (SELECT MAX(id) FROM users);
  batch_size int := 1000;
  rows_updated int;
BEGIN
  WHILE current_id <= max_id LOOP
    -- the UPDATE by `id` range
    UPDATE users
    SET email = scrub_email(email)
    WHERE id >= current_id
    AND id < current_id + batch_size;

    GET DIAGNOSTICS rows_updated = ROW_COUNT;

    COMMIT;
    RAISE NOTICE 'current_id: % - Number of rows updated: %',
    current_id, rows_updated;

    current_id := current_id + batch_size + 1;
  END LOOP;
END;
$$;

-- Call the Procedure
CALL scrub_batches();
```

You may paste this procedure directly into psql for Rideshare, and note that it will run right away, updating batches of users rows.

A WHILE loop is being used which is similar to a While loop in Ruby. It will run until a condition is met.

The GET DIAGNOSTICS rows_updated = ROW_COUNT; clause captures the number of rows being updated in one loop iteration. The UPDATE statement updates all of the rows that are matched inside of the id column range. This will update 1000 rows with a batch size of 1000.

Values are printed out in a similar way to puts in Ruby using the RAISE NOTICE clause. The % symbol here is similar to String interpolation in Ruby.

Procedures can be copied directly into psql to run. However, similar to database functions, you'll want to treat procedures you use with your appli-